

Reviewer Pack — Matroid Matching of Term Co-Occurrence Graph Bounds k-CLIQUE...

Entry 02c57a92c88c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-16 20:46:58 UTC

Matroid Matching of Term Co-Occurrence Graph Bounds k-CLIQUE Monotone DNF

Entry ID: 02c57a92c88c **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-16 20:46:58 UTC

1. Conjecture

Field A (mathematical branch): Matroid matching theory (Lovász 1980 polymatroid matching theorem, applied to term co-occurrence graphs of monotone DNFs) **Field B** (complexity object): Monotone DNF size and Karchmer-Wigderson protocols for the k-CLIQUE indicator

Statement:

For a monotone DNF $F = T_1 \vee \dots \vee T_m$ on n variables, build the *term co-occurrence graph* $H_F = ([n], E_F)$ with $E_F = \{\{u, v\} : \exists i, \{u, v\} \subseteq T_i\}$, and define $\mu(F) = \text{match}(H_F) / \lceil \log_2(1+m) \rceil$, where $\text{match}(H_F)$ is the size of a maximum matching in H_F . We conjecture: (i) μ is subadditive under DNF conjunction-expansion, $\mu(F \wedge G) \leq \mu(F) + \mu(G)$; (ii) $\mu(F) \leq 4$ for every monotone DNF F of ‘log-width’ type, namely $m \leq n^c$ and $\max_i |T_i| \leq c \cdot \log_2 n$ for any fixed $c \geq 1$; (iii) for the canonical k-CLIQUE DNF $F_{\{v, k\}}$ on v vertices with $k = \lceil \log_2 v \rceil$ (variables = the $C(v, 2)$ edges), $\mu(F_{\{v, k\}}) \geq v / (2 \cdot (\log_2 v)^2)$, i.e. unbounded as $v \rightarrow \infty$.

Rationale (proposer’s reasoning):

Karchmer-Wigderson translates monotone formula size into communication on terms; matroid matching of the co-occurrence graph H_F is a representation-level polymatroid invariant capturing how ‘expansive’ the term overlap pattern is. Log-width poly-DNFs use few variables per term, keeping matchings tame, while every pair of vertices lies in some k-clique-term for k-CLIQUE, forcing H_F to be the complete graph K_v and matchings to scale linearly. Because μ reads the *syntactic* term structure rather than the truth table, it sidesteps Razborov-Rudich naturalness while staying squarely inside the KW monotone-protocol techniques.

Taxonomy category: KARCHMER_WIGDERSON (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): cef7fa4bd00b4d5f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across the three pre-registered tests using fixed random seeds 1-30: (i) subadditivity $\mu(F \wedge G) \leq \mu(F) + \mu(G)$ holds in $\geq 28/30$ $n=12$ pairs; (ii) $\mu(F) \leq 4$ holds in $\geq 27/30$ trials at EACH $n \in \{16, 25, 36\}$; (iii) $\mu(F_{\{v, k\}}) \geq v/(2 \cdot (\log_2 v)^2)$ for ALL $v \in \{6, 8, 10, 12, 14\}$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - monotone DNF lower bounds k-clique Karchmer Wigderson - polymatroid matching Lovász monotone formula complexity - term co-occurrence graph matching DNF size lower bound

Top relevant hits considered: - [<http://arxiv.org/abs/2107.05128v2>] Karchmer-Wigderson Games for Hazard-free Computation - [<http://arxiv.org/abs/1503.06447v1>] Sum-of-squares lower bounds for planted clique - [<http://arxiv.org/abs/2405.01069v1>] Oriented Ramsey numbers of graded digraphs - [<http://arxiv.org/abs/2505.06446v1>] Structured Prediction with Abstention via the Lovász Hinge - [<http://arxiv.org/abs/1102.2932v2>] Applications of Monotone Rank to Complexity Theory - [<http://arxiv.org/abs/2309.13639v1>] A deletion-contraction formula and monotonicity properties for the polymatroid Tutte polynomial - [<http://arxiv.org/abs/1406.7168v1>] Co-occurrence matrices of time series applied to literary works - [<http://arxiv.org/abs/1806.02734v3>] Spectral lower bounds for the orthogonal and projective ranks of a graph

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.4s

5.1 Generated Python source

```
import sys
import random
import math
import itertools
from collections import defaultdict

def generate_dnf(n, m, max_term_size, seed):
    random.seed(seed)
    terms = []
    for _ in range(m):
        term_size = random.randint(2, min(max_term_size, n))
        term = random.sample(range(1, n + 1), term_size)
        terms.append(term)
    return terms

def generate_dnf_pair(n, seed):
```

```

random.seed(seed)
m = int(n ** 1.5)
max_term_size = 2 * math.ceil(math.log2(n))
terms1 = generate_dnf(n, m, max_term_size, seed)
terms2 = generate_dnf(n, m, max_term_size, seed + 1)
return terms1, terms2

def build_co_occurrence_graph(terms):
    graph = defaultdict(set)
    for term in terms:
        for u, v in itertools.combinations(term, 2):
            graph[u].add(v)
            graph[v].add(u)
    return graph

def find_maximum_matching(graph):
    matching = {}
    for u in graph:
        if u not in matching:
            for v in graph[u]:
                if v not in matching:
                    matching[u] = v
                    matching[v] = u
                    break
    return len(matching) // 2

def compute_mu(terms):
    if not terms:
        return 0.0
    graph = build_co_occurrence_graph(terms)
    max_matching = find_maximum_matching(graph)
    m = len(terms)
    denominator = math.ceil(math.log2(1 + m))
    if denominator == 0:
        return 0.0
    return max_matching / denominator

def generate_canonical_k_clique_dnf(v, k):
    terms = []
    for edges in itertools.combinations(range(1, v + 1), k):
        term = []
        for i, j in itertools.combinations(edges, 2):
            term.append(i * v + j)
        terms.append(term)
    return terms

def run_trial(seed):
    random.seed(seed)
    n_values = [16, 25, 36]
    v_values = [6, 8, 10, 12, 14]
    metric_values = []
    conjecture_holds = True
    counterexample = ""

    # Test (i): Subadditivity
    n = 12

```

```

m = int(n ** 1.5)
max_term_size = 2 * math.ceil(math.log2(n))
terms1, terms2 = generate_dnf_pair(n, seed)
expanded_terms = [list(set(t1) | set(t2)) for t1, t2 in itertools.product(terms1, terms2)]
mu_fg = compute_mu(expanded_terms)
mu_f = compute_mu(terms1)
mu_g = compute_mu(terms2)
if mu_fg > mu_f + mu_g:
    conjecture_holds = False
    counterexample = f"Subadditivity failed: mu(FAG)={mu_fg} > mu(F)+mu(G)={mu_f + mu_g}"

# Test (ii): Log-width bound
for n in n_values:
    m = int(n ** 1.5)
    max_term_size = 2 * math.ceil(math.log2(n))
    terms = generate_dnf(n, m, max_term_size, seed)
    mu = compute_mu(terms)
    if mu > 4:
        conjecture_holds = False
        counterexample = f"Log-width bound failed: mu={mu} > 4 for n={n}"

# Test (iii): k-CLIQUE bound
for v in v_values:
    k = math.floor(math.log2(v))
    terms = generate_canonical_k_clique_dnf(v, k)
    mu = compute_mu(terms)
    lower_bound = v / (2 * (math.log2(v) ** 2))
    if mu < lower_bound:
        conjecture_holds = False
        counterexample = f"k-CLIQUE bound failed: mu={mu} < {lower_bound} for v={v}"

return {
    "metric_name": "mu",
    "metric_value": mu_fg if conjecture_holds else 0.0,
    "instances_tested": len(n_values) + len(v_values) + 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [i for i in range(30)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    metric_values = [r["metric_value"] for r in results]
    mean_metric = sum(metric_values) / len(metric_values)
    std_metric = math.sqrt(sum((x - mean_metric) ** 2 for x in metric_values) / len(metric_values))
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])

```

```

        counterexample = next(r["counterexample"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

TRIAL: {'metric_name': 'mu', 'metric_value': 0.0, 'instances_tested': 9, 'conjecture_holds': False, '
CLIQUE bound failed: mu=0.0 < 0.4489652833957362 for v=6'}
TRIAL: {'metric_name': 'mu', 'metric_value': 0.0, 'instances_tested': 9, 'conjecture_holds': False, '
CLIQUE bound failed: mu=0.0 < 0.4489652833957362 for v=6'}
TRIAL: {'metric_name': 'mu', 'metric_value': 0.0, 'instances_tested': 9, 'conjecture_holds': False, '
CLIQUE bound failed: mu=0.0 < 0.4489652833957362 for v=6'}
TRIAL: {'metric_name': 'mu', 'metric_value': 0.0, 'instances_tested': 9, 'conjecture_holds': False, '
CLIQUE bound failed: mu=0.0 < 0.4489652833957362 for v=6'}
TRIAL: {'metric_name': 'mu', 'metric_value': 0.0, 'instances_tested': 9, 'conjecture_holds': False, '
CLIQUE bound failed: mu=0.0 < 0.4489652833957362 for v=6'}
TRIAL: {'metric_name': 'mu', 'metric_value': 0.0, 'instances_tested': 9, 'conjecture_holds': False, '
CLIQUE bound failed: mu=0.0 < 0.4489652833957362 for v=6'}
TRIAL: {'metric_name': 'mu', 'metric_value': 0.0, 'instances_tested': 9, 'conjecture_holds': False, '
CLIQUE bound failed: mu=0.0 < 0.4489652833957362 for v=6'}
TRIAL: {'metric_name': 'mu', 'metric_value': 0.0, 'instances_tested': 9, 'conjecture_holds': False, '
CLIQUE bound failed: mu=0.0 < 0.4489652833957362 for v=6'}
TRIAL: {'metric_name': 'mu', 'metric_value': 0.0, 'instances_tested': 9, 'conjecture_holds': False, '
CLIQUE bound failed: mu=0.0 < 0.4489652833957362 for v=6'}
TRIAL: {'metric_name': 'mu', 'metric_value': 0.0, 'instances_tested': 9, 'conjecture_holds': False, '
CLIQUE bound failed: mu=0.0 < 0.4489652833957362 for v=6'}
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_eb505197.py", line 137, in <module>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                        ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_eb505197.py", line 137, in <genexpr>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                        ~~~~~^
KeyError: 'seed'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The test reports conjecture_holds=False with an explicit counterexample: for the k-CLIQUE DNF at v=6, $\mu=0.0 < 0.449$, violating clause (iii) of the pre | next: Recompute $\mu(F_{\{v,k\}})$ by hand for small v to determine whether the lower bound $v/(2 \cdot (\log_2 v)^2)$ is fundamentally wrong or whether the matching/term-graph construction in the test mis-encodes the k-CLIQUE DNF.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	392843
2	preregistration	claude_max	opus	0	0	6197
3	novelty	claude_max	opus	0	0	5905
4	novelty	claude_max	opus	0	0	10504
5	test_gen	mistral	codestral-latest	0	0	307961
6	test_gen	mistral	codestral-latest	0	0	311876
7	test_gen	mistral	codestral-latest	0	0	309486
8	test_gen	mistral	codestral-latest	0	0	308860
9	judge	claude_max	opus	0	0	5190

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 1658821 ms total latency. Provider mix: {'claude_max': 5, 'mistral': 4}

(full prompt+response transcripts available in research/audit/02c57a92c88c.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/02c57a92c88c.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/02c57a92c88c.tar.gz (if generated)